

Financial Time-Series MLOps

Predicting Next-Day Market-Open Direction
for NSE F&O Stocks Using OHLCV and News Sentiment

Kalp Garg (DA25M573)
Aditya Gaurav (DA25G502)
Allenki Laxmi (DA25M541)

MLOps Course End-Term Project

August 23, 2026

Contents

1	Problem Statement	3
2	Dataset Description	3
2.1	Stock Universe	3
2.2	OHLCV Price Data	3
2.3	News Headlines	4
2.4	Data Contracts	4
3	System Architecture	4
3.1	High-Level Data Flow	5
3.2	Orchestration	5
3.3	Data and Model Versioning	5
4	Data Pipeline	5
4.1	Scrapers	5
4.2	PySpark Processing	6
4.3	Airflow Inference DAG	6
5	Deployment Strategy	6
5.1	Serving Architecture: Two Paths, One Scoring Module	6
5.2	FastAPI Endpoints	7
5.3	Prediction Store	7
5.4	Docker Containerisation	7
5.5	Artifacts and Reproducibility	7
6	Monitoring Strategy	8
6.1	Prometheus Metrics	8
6.2	Scrape Configuration	8
6.3	Example PromQL Queries	8
6.4	Prediction Logging and Health Checks	8
6.5	Drift Detection (Designed; Future Work)	9

7	CI/CD Implementation	9
7.1	Continuous Integration (GitHub Actions)	9
7.2	Test Suite	9
7.3	Reproducibility	10
7.4	Continuous Delivery (Extension)	10
8	ML Modelling	10
8.1	Cross-Validation Results	10
8.2	Cross-Validation Results	10
8.3	Per-Class Performance	10
8.4	Feature Importance	11
8.5	Discussion	11
9	Challenges Faced	11
10	Future Improvements	12

1 Problem Statement

This project builds an end-to-end MLOps pipeline to predict the **next-day market-open price direction** (Bullish / Bearish / Neutral) for 50 NSE Futures & Options (F&O) constituent stocks.

The prediction target is a three-class variable derived from the return between the next day's 09:15 AM opening price and its closing price. A return exceeding +0.3% is labelled *Positive*, below -0.3% is *Negative*, and anything in between is *Neutral*.

The system ingests two complementary data streams:

1. **OHLCV price data** — 15-minute intraday bars from TradingView, aggregated into daily candles with rolling technical indicators.
2. **Financial news headlines** — scraped from Groww using Crawl4AI, enriched with FinBERT (ProsusAI/finbert) sentiment probabilities and 768-dimensional CLS embeddings, then PCA-reduced and aggregated with exponential-decay rolling windows.

The merged feature set (95 features) feeds an XGBoost / LightGBM classifier trained with TimeSeriesSplit cross-validation and tracked via MLflow. The best model is served through a containerised FastAPI endpoint and monitored with Prometheus metrics.

Minimum expected scope covered.

- Automated pipeline using **Airflow** (inference and training DAGs).
- Additional data-engineering tool: **PySpark** (headline cleaning, pre-open merge).
- Data preprocessing workflow (scraping → Spark cleaning → FinBERT → feature engineering).
- Training and comparison of **two models** (XGBoost and LightGBM).

2 Dataset Description

All data is stored as DVC-tracked CSV files in the repository's `data/` directory.

2.1 Stock Universe

The universe comprises **50 NSE F&O stocks** listed in `data/stock_list.csv`. Each row maps a company name (`Stock_name`) to its TradingView identifier, Groww slug, and NSE symbol.

2.2 OHLCV Price Data

Table 1: OHLCV data files		
File	Granularity	Description
<code>merged_ohlc_15min.csv</code>	15-minute	Intraday bars for all 50 stocks. Columns: <code>symbol</code> , <code>datetime</code> , <code>open</code> , <code>high</code> , <code>low</code> , <code>close</code> , <code>volume</code> .
<code>ohlc_data/<stock>.csv</code>	15-min / daily	Per-stock raw files (daily and 15-min), merged into the above.
<code>preopen_csv/</code>	Point-in-time	NSE F&O pre-open auction data, one dated CSV per trading day.

OHLCV data is fetched via the TradingView data feed (`tvDatafeed` library). Each stock has up to 10 000 historical 15-minute bars (~2 years). The scraper supports incremental updates, fetching only bars newer than the latest timestamp in the existing CSV.

2.3 News Headlines

Table 2: Headline data files

File	Description
stock_news/headlines.csv	Raw scraped headlines (Crawl4AI from Groww). Columns: headline_id, symbol, published_at, source, headline, article_url, scraped_at.
headlines.csv	Cleaned and deduplicated headlines (PySpark output, DVC-tracked).
headlines_enriched.csv	FinBERT-enriched: positive_prob, negative_prob, neutral_prob, sentiment_score, sentiment_label.
headlines_enriched.embeddings.npy	768-d FinBERT CLS embeddings (NumPy binary).
news_features.csv	PCA-reduced (50-d) + 7-day rolling aggregated sentiment features per (symbol, date).

2.4 Data Contracts

All inter-role data exchange is governed by dataclass schemas defined in `shared/schemas/data_contract.py`:

```
1 @dataclass
2 class PriceRecord:
3     symbol: str          # e.g. "Reliance Industries"
4     date: str            # ISO-8601 date
5     open: float
6     high: float
7     low: float
8     close: float
9     volume: int
10
11 @dataclass
12 class HeadlineRecord:
13     headline_id: str
14     symbol: str
15     published_at: str # ISO-8601 datetime
16     source: str
17     headline: str
18     author: Optional[str] = None
19     body_snippet: Optional[str] = None
```

Listing 1: Core data-contract schemas

3 System Architecture

The project is organised into three roles, each with its own directory, dependencies, and Airflow DAGs:

Module	Source	Output	Notes
headline_scraper	Groww (Crawl4AI)	stock_news/headlines.csv	Async headless browser; MD5-based <code>headline_id</code> for dedup
ohlc_scraper	TradingView	merged_ohlc_15min.csv	Up to 10k 15-min bars/stock; per-stock CSVs merged
nse_preopen_scraper	NSE API	preopen_csv/*.csv	F&O pre-open auction; exponential-backoff retries
finbert_inference	ProsusAI/finbert	enriched CSV + .npy	Batched sentiment probs + 768-d CLS embeddings; $s = (p_+ - p_-)(1 - p_0)$

4.2 PySpark Processing

Two PySpark jobs run in `local[*]` mode. **clean_headlines** applies a UDF (lowercase, strip HTML, collapse whitespace), deduplicates by `headline_id` via a window function, and writes `data/headlines.csv`. **merge_preopen** maps NSE tickers to stock names via a Spark join on `stock_list.csv`, creates synthetic 09:15 OHLCV bars from the pre-open data, deduplicates, and appends to `merged_ohlc_15min.csv`. A third job, **trim_for_inference**, filters the full CSVs to the minimal date window (45-day OHLCV, 10-day headlines) needed by the Docker inference container.

4.3 Airflow Inference DAG

The DAG `financial_ts_inference_pipeline` runs daily at 08:30 IST with seven tasks: scrape headlines + OHLCV (parallel) → scrape pre-open → Spark clean + merge (parallel) → Spark trim → Docker inference (`batch_score.py`, trimmed data mounted read-only) → DVC push. Each task runs its module via `subprocess.run()` for import isolation. Setting `SKIP_SCRAPERS=1` disables live scraping for testing.

5 Deployment Strategy

The trained model is served as a containerised **FastAPI** application backed by **PostgreSQL** and orchestrated with the rest of the system via **Docker Compose**. What is served is not a single estimator but the five-stage pipeline (FinBERT → PCA → news aggregation → OHLCV features → XGBoost) producing a three-class direction (Negative / Neutral / Positive) with class probabilities.

5.1 Serving Architecture: Two Paths, One Scoring Module

Predictions are produced through two paths that share a *single* scoring implementation (`app/pipeline.py`) — a deliberate choice that prevents batch/online and train/serve skew:

- **Batch scoring (authoritative)**. Once per trading day the Airflow inference DAG runs `batch_score.py`, scoring all 50 constituents and writing the results to the database. This is the source of truth served to consumers.
- **On-demand API (POST /predict)**. A single symbol is scored live from data supplied in the request, so the model is genuinely *served*, not merely executed inside a DAG.

Why batch-first. FinBERT inference over the news of all constituents is compute-heavy (seconds-to-minutes on CPU) and cannot run synchronously inside an HTTP request without timeouts; predictions are also deterministic per trading day over a fixed universe. Precomputing once and serving reads decouples model execution from request serving, yields constant-time reads, and produces a durable, auditable prediction history.

5.2 FastAPI Endpoints

Endpoint	Purpose
POST /predict	Score one symbol on demand from raw news + OHLCV; records the inference
GET /predictions	Read the authoritative daily batch predictions (consumer endpoint)
GET /health	Liveness probe: model-loaded status, model version, database connectivity
GET /metrics	Prometheus metrics (Section 6)

Request and response payloads are validated with **Pydantic**, so a mismatch between the data pipeline and the model fails loudly at the boundary (HTTP 422) rather than producing a silent, incorrect prediction.

5.3 Prediction Store

Predictions are written to two deliberately separate tables:

- **pipeline_predictions** — the authoritative daily batch output, one row per (symbol, date) enforced by a **UNIQUE** constraint and written with an **upsert**. This makes the Airflow scoring task **idempotent** (a retry updates rather than duplicates) and carries **run_id** to trace each row to a pipeline run.
- **api_predictions** — ad-hoc POST /predict inferences, carrying **request_id** and **latency_ms**, kept separate so ad-hoc calls cannot pollute the authoritative history.

Both tables store the three class probabilities, **model_version**, and a timezone-aware timestamp. SQLite is used for local development and PostgreSQL in the deployed stack; the same code path (including the dialect-agnostic upsert) works on both.

5.4 Docker Containerisation

A single multi-stage **Dockerfile** defines three purpose-built images so the API's dependencies never collide with Airflow's or MLflow's:

- **api** — the FastAPI service on **python:3.12-slim**, with **CPU-only PyTorch** (installed from PyTorch's CPU index to avoid a multi-GB CUDA build), the **FinBERT weights baked in** at build time (fully offline at runtime), a **non-root** user, and a **HEALTHCHECK** on /health.
- **airflow** — the orchestrator image (official Airflow + PySpark + DVC).
- **mlflow** — the experiment-tracking server.

A root **docker-compose.yml** brings up the whole system with a single **docker compose up** — the API, PostgreSQL, Prometheus, MLflow, and Airflow (webserver + scheduler) — with the API gated on the database's health check so the first request never races an unready database. The exposed UIs are the API docs (:8000/docs), Prometheus (:9090), MLflow (:5000), and Airflow (:8080).

5.5 Artifacts and Reproducibility

The model artifacts (PCA, symbol encoder, feature-column list, XGBoost model) are **versioned with DVC** and **mounted into the container at runtime** (**ARTIFACTS_DIR=/app/models**) rather than baked into the image, so a retrained model is served by swapping files and restarting — no rebuild. To guarantee reproducibility, the serving environment is pinned to the exact

versions the artifacts were trained with (Python 3.12, scikit-learn 1.9.0, xgboost 3.4.1); a mismatch would risk silent, incorrect deserialisation. Every stored prediction is stamped with `MODEL_VERSION` to tie it to the model that produced it.

6 Monitoring Strategy

Monitoring combines **Prometheus metrics**, **structured prediction logging**, and **health checks**, covering three of the options the project allows (Prometheus, API latency, prediction logging).

6.1 Prometheus Metrics

Every prediction updates four Prometheus signals, exposed at `GET /metrics` in the standard text exposition format:

Metric	Type	Signal
<code>prediction_requests_total</code>	counter	request count and success/error split
<code>inference_latency_seconds</code>	histogram	inference-latency distribution
<code>predictions_by_direction_total</code>	counter	predicted-class distribution
<code>prediction_confidence</code>	histogram	confidence-score distribution

The confidence histogram buckets are deliberately fine-grained at the top end (`...`, `0.9`, `0.95`, `0.99`, `1.0`) because the model's confidence skews very high; evenly spaced buckets would collapse the distribution into a single bin.

6.2 Scrape Configuration

A Prometheus service (configured in `monitoring/prometheus/prometheus.yml`) scrapes the API every 15 seconds on the Compose network and is reachable at `localhost:9090`.

6.3 Example PromQL Queries

```
1 # Predicted-class mix
2 sum by (direction) (predictions_by_direction_total)
3
4 # 95th-percentile inference latency
5 histogram_quantile(0.95, sum by (le) (rate(inference_latency_seconds_bucket[5m])
6   ))
7
8 # Error rate
9 sum(rate(prediction_requests_total{outcome="error"}[5m]))
```

Listing 3: Representative monitoring queries

6.4 Prediction Logging and Health Checks

Each `POST /predict` call emits a structured log record (request id, symbol, predicted class, confidence, latency, model version) and is persisted to the `api_predictions` table with its measured `latency_ms`, giving both operational telemetry and an auditable per-request trail. The `GET /health` endpoint reports model and database status and backs the container health check.

6.5 Drift Detection (Designed; Future Work)

A feature-drift design was specified but not yet implemented: comparing the serving-time feature distribution against a training-time baseline using the **Population Stability Index (PSI)** per feature (thresholds 0.1 / 0.25), plus predicted-class drift, surfaced as Prometheus gauges and a `/drift` endpoint. It is deferred pending the training feature matrix from the modelling role.

7 CI/CD Implementation

7.1 Continuous Integration (GitHub Actions)

A GitHub Actions workflow (`.github/workflows/ci.yml`) acts as a **merge gate**: on every pull request to `main` and every push to `main`, it sets up Python 3.12, installs a curated test-dependency set, and runs the full suite with `pytest`. With branch protection requiring this check, a failing suite blocks the merge.

```
1 on:
2   pull_request: { branches: [main] }
3   push:         { branches: [main] }
4 jobs:
5   test:
6     runs-on: ubuntu-latest
7     steps:
8       - uses: actions/checkout@v4
9       - uses: actions/setup-python@v5
10        with: { python-version: "3.12", cache: pip }
11       - run: pip install -r requirements-test.txt
12       - run: pytest -q
```

Listing 4: CI workflow (abridged)

The workflow installs only the **light dependencies the tests need** (`requirements-test.txt`) rather than every role’s full requirements: the heavy libraries (PyTorch, Transformers, Airflow, Crawl4AI) are unnecessary because FinBERT is stubbed in tests and other heavy imports are guarded with `importorskip`. This keeps CI fast and reliable.

7.2 Test Suite

The suite (39 tests) spans the whole project:

Scope	Coverage
Shared / root	Cross-role data contract and configuration helpers
Role 1	PySpark text-cleaning logic
Role 2	Feature engineering (daily aggregation, next-day 09:15 shift, three-class target), model factories, evaluation metrics, and a model-artifact contract test (95 features, 3 classes, 50 symbols, 768→50 PCA)
Role 3	API endpoints, request validation (422 cases), batch idempotency, and the Prometheus metrics

Tests that require the DVC-tracked model artifacts **skip automatically when the artifacts are absent** (e.g. in CI without a `dvc pull`), so the pipeline stays green while still running the full model-dependent tests locally where the artifacts are present. The `.pkl` interface guarantee is anchored by the role 2 artifact-contract test.

7.3 Reproducibility

The project environment is defined by `pyproject.toml` (with `uv.lock`) and per-role `requirements.txt` files; data and models are versioned with DVC; and the serving library versions are pinned to match the training environment. CI runs on the same Python version (3.12) as the deployed container.

7.4 Continuous Delivery (Extension)

The current pipeline implements the CI (test-gate) half. The delivery half — a job that builds the Docker image on merges to `main` — is a straightforward extension: the API image builds without the artifacts (they are mounted at runtime), so it is feasible in CI, with the only cost being the FinBERT download during the build.

8 ML Modelling

8.1 Cross-Validation Results

The performance of the developed machine learning models was evaluated using time-series cross-validation. XGBoost and LightGBM were evaluated using the same input features and target variable to maintain consistency in the comparison. Macro F1-score was considered as the primary evaluation metric, as it provides balanced consideration to all stock movement classes.

8.2 Cross-Validation Results

The cross-validation results indicate that both XGBoost and LightGBM achieved comparable performance on the stock movement classification task. XGBoost achieved a macro F1-score of 0.6631, while LightGBM achieved a macro F1-score of 0.6572.

Table 4: Cross-Validation Performance

Model	Macro F1-Score
XGBoost	0.6631
LightGBM	0.6572

The difference between the models is relatively small, indicating that both gradient-boosting approaches are capable of learning meaningful patterns from the combined sentiment and market features. XGBoost achieved the higher macro F1-score and therefore provided the best overall cross-validation performance among the evaluated models.

8.3 Per-Class Performance

The prediction problem consists of negative, neutral, and positive stock movement classes. In addition to overall performance, class-wise evaluation is useful for understanding how effectively the models distinguish between the different types of market movement.

Precision, recall, and F1-score can be considered independently for each movement class. Macro F1-score summarizes this class-level performance by giving equal importance to each class, making it suitable for comparing the models without allowing the performance of a more frequently occurring class to dominate the overall evaluation.

The predicted class probabilities also provide information about the confidence of individual predictions. The highest probability among the movement classes is considered as the confidence associated with the model’s predicted class.

8.4 Feature Importance

Feature importance analysis provides insight into the variables that contribute to the predictions generated by the models. The final feature set contains information obtained from both financial news and historical market data.

The news-based features include FinBERT sentiment information, sentiment aggregation features, and PCA-reduced contextual embeddings. The market-based features include price returns, lagged returns, volatility, moving averages, price deviations, trading-volume information, and intraday market behaviour.

Analyzing feature importance helps determine whether the models primarily depend on historical market behaviour, information extracted from financial news, or a combination of both. It also improves the interpretability of the prediction pipeline by identifying the features that have greater influence on the classification decisions.

8.5 Discussion

The experimental results demonstrate that both XGBoost and LightGBM can be used effectively within the proposed financial prediction pipeline. The close cross-validation performance of the models suggests that both algorithms are able to capture useful nonlinear relationships between sentiment-derived features and historical market behaviour.

XGBoost achieved a slightly higher macro F1-score than LightGBM and therefore performed better according to the primary evaluation metric. However, the small difference between their results indicates that the overall predictive capability of the two models is comparable.

The use of time-series cross-validation ensures that the chronological nature of financial data is preserved during model evaluation and reduces the risk of look-ahead bias. The combination of FinBERT-derived news information and engineered OHLCV features enables the models to incorporate both textual market information and historical price behaviour.

MLflow experiment tracking further supports the evaluation process by maintaining the hyperparameters, fold-level metrics, average performance, and model artifacts associated with each experiment. This provides a reproducible and traceable modelling workflow and enables systematic comparison of the different model configurations.

Based on the cross-validation results, XGBoost provides the highest overall macro F1-score among the evaluated models and can therefore be considered the preferred model based on the current evaluation results.

9 Challenges Faced

1. **Heavy dependencies in CI.** Running the full cross-role dependency set in CI is slow and fragile; this was solved with a curated test-dependency set, a stubbed FinBERT, and `importorskip` guards.
2. **Model artifacts unavailable in CI.** The artifacts are DVC-tracked and the remote is unreachable from CI; model-dependent tests were made to skip gracefully, with the `.pk1` interface contract guaranteed by a modelling-role test.
3. **FinBERT CPU latency.** FinBERT inference on CPU takes several seconds per batch of 32 headlines, making synchronous single-request scoring marginal for user-facing latency. The batch-first architecture mitigates this for the daily pipeline, but real-time on-demand scoring remains slow without GPU or model distillation.
4. **Scraper fragility.** Both Crawl4AI (headless-browser scraping of Groww) and the NSE pre-open API are subject to rate-limiting, layout changes, and intermittent failures. Exponential-

backoff retries and incremental updates reduce the impact but do not eliminate it; a prolonged outage produces stale or empty input data.

10 Future Improvements

1. **GPU / ONNX FinBERT.** Exporting the FinBERT encoder to ONNX or moving inference to a GPU-backed container would reduce per-batch latency from seconds to milliseconds, enabling true real-time single-request scoring.
2. **Additional data sources.** Options-chain data (open interest, implied volatility), FII/DII flow data, and macroeconomic indicators would enrich the feature set and may improve directional accuracy.
3. **Automated retraining pipeline.** A scheduled retraining DAG with automatic model promotion (champion/challenger comparison using MLflow’s model registry) when the validation macro F1 exceeds the current champion.
4. **Prediction frontend.** A lightweight dashboard (Streamlit or React) over `GET /predictions` to visualise daily batch predictions, confidence distributions, and historical accuracy.